

ProcessPilot AI ? From Problem to Production

A Full-Stack AI Engineering & Problem-Solving Case Study

Problem -> Requirements -> Architecture -> Implementation -> Debugging -> Iteration -> Testing -> Deployment

Author: Aritra Das | Version: 1.0 | Date: September 2026

1. The Original Problem

I started with a human engineering problem, not a technology stack. Organizational knowledge is fundamentally fragmented. Documents (policies, SOPs) live in static files. Meetings produce action items that disappear the moment the call ends. Operational relationships (who reports to whom, what department owns what) live in HR systems.

What made this technically interesting was the isolation of these silos. I wanted to explore whether I could build a system that brings organizational knowledge, conversations, documents, and operational context together, making it queryable through a single AI interface.

2. Problem Decomposition

I broke this massive idea into smaller engineering questions:

- Document Knowledge: How do I ingest documents and search them semantically?
- Meeting Knowledge: How do I turn unstructured conversational transcripts into tracked tasks?
- Relational Knowledge: How do I answer questions about organizational structure?
- Security: How do I protect sensitive PII before an AI ever sees it?
- Access: How do I ensure users only see their department's data?
- Resilience: How does the AI recover when an external LLM provider goes down?
- Infrastructure: How do I deploy this on resource-constrained free-tier environments?

3. Requirements & The PRD

Starting development immediately would have led to uncontrolled feature growth. I wrote a PRD to map problems to requirements:

- Problem: Fragmented documents -> Requirement: Centralized RAG pipeline.
- Problem: Lost meeting actions -> Requirement: Transcript analysis and task extraction.
- Problem: Relational queries fail in vector search -> Requirement: Knowledge Graph.
- Problem: AI hallucination -> Requirement: Grounded retrieval and strict JSON schemas.

4. PRD to TRD (Technical Decisions)

I translated product requirements into technical architecture. I chose React 19 for a fast frontend. For the backend, I needed high concurrency for LLM streaming, so I chose FastAPI (async Python). Because I needed relational data (RBAC), vectors (RAG), and graph relationships without managing three separate databases, I chose PostgreSQL (Supabase) using the pgvector extension.

5. Database Design & Foundation

Reasoning

What information does the system need to remember? Users belong to Departments. Documents are broken into Chunks, which have Embeddings. Meetings generate Tasks. Interactions create Agent Logs and LLM Usage metrics.

Implementation

I designed an 18-table schema in SQLAlchemy. Critical access boundaries (ABAC) were implemented directly in the relational schema by tying Documents, Users, and Tasks to department_ids.

6. Document Ingestion & PII Security

Problem

How do I turn arbitrary organizational documents into searchable AI information while protecting sensitive data?

Initial Approach & Limitation

Initially, I planned to extract text using PyMuPDF and directly embed it. However, I realized that HR documents contain SSNs and credit card numbers. Sending this raw text to external LLM APIs (OpenAI/Groq) creates massive security exposure.

Implementation Change (PII Redaction)

I introduced a security layer *before* chunking. I built pii_redactor.py, using Regex patterns to scrub sensitive data, replacing it with tokens like [REDACTED_SSN].

Trade-off

Regex is fast and doesn't require heavy ML models (like Presidio), which saves memory on free-tier servers. However, it struggles with edge-case formatting. The trade-off was accepting minor leakage risk in exchange for system stability and low memory footprint.

7. RAG & Retrieval Evolution

Problem

Semantic retrieval (Cosine Similarity via pgvector) works great for 'how to apply for leave'. It completely fails for keyword-heavy queries like 'Document ID HR-992-B'.

Investigation

Vector embeddings compress meaning, destroying exact character matching. I needed a keyword signal.

Implementation Change

I added BM25Okapi to create a Hybrid Retrieval system. The system now retrieves dense vectors from Postgres and sparse keyword matches locally.

Future Limitation

Currently, the scores are combined loosely. Reciprocal Rank Fusion (RRF) is planned but NOT IMPLEMENTED to mathematically normalize these scores.

8. Multi-Agent Orchestration

Problem

A single RAG prompt cannot answer every question. If a user asks 'Who wrote the onboarding policy?',

standard RAG fails.

Implementation

I built a Hierarchical Routing Architecture. A CEOAgent analyzes the user's query intent and routes it to specialized sub-agents: SearchAgent (RAG), GraphAgent (SQL Graph), or SOPAgent (Policies).

Clarification

This is specialized agent orchestration (delegation routing). It is NOT an unrestricted autonomous tool-calling loop. Constraining the agents made the system faster and more predictable.

9. Knowledge Graph Architecture Evolution

Before

To map relational data, I initially used NetworkX, storing the graph as an in-memory JSON file.

Problem Discovered

In a stateless, horizontally scalable environment (like Render), in-memory JSON files are wiped constantly. The graph kept resetting.

Implementation Change

I migrated the entire graph to SQL. I created `kg_nodes` and `kg_edges` tables in PostgreSQL. The backend now queries standard relational joins to reconstruct graph relationships.

Trade-off

I lost the ability to run complex native graph traversal algorithms (like PageRank), but I gained infinite horizontal scalability, persistence, and reduced infrastructure complexity.

10. Meeting Intelligence & Real-Time Tasks

Problem

Meeting transcripts contain tasks. I prompted the LLM to return JSON arrays of tasks. However, LLMs often hallucinate conversational filler ('Here is the JSON you requested: [...]').

Implementation Change

I bypassed the hallucinations by implementing strict Regex extraction (`re.search(r'\{.*\}')`) to isolate the payload. These tasks are then saved to the database.

Real-Time Updates

I added WebSockets (`ws.py`) so that the moment the AI parses a task, it pushes a notification directly to the assigned user's React frontend without requiring a page refresh.

11. Real Engineering Problems I Encountered

Problem 1: Groq Model Deprecations (404s)

Symptom: The AI suddenly stopped responding entirely, throwing HTTP 500s.

Investigation: Groq had deprecated the specific llama3 model string I hardcoded.

Fix: I rewrote `llm_client.py` to catch 404 errors, dynamically query Groq's `/models` endpoint for active models, and automatically retry. The system is now immune to model deprecations.

Problem 2: Postgres Timezone Crashes

Symptom: The frontend Analytics page crashed entirely.

Investigation: `asyncpg` returns timezone-aware datetimes from Postgres. Python's `sorted()` function crashed when comparing these against naive datetimes generated locally.

Fix: I implemented a strict UTC normalization utility in `analytics.py`. All dates are converted to UTC before any sorting occurs.

12. Observability & Testing

Observability

Standard CRUD logs (200 OK) are useless for AI. If the AI hallucinates, the server still returns 200. I built dedicated tables: `ai_failures` (tracking API timeouts), `llm_usage` (tracking token costs), and `agent_logs` (tracking routing decisions).

Testing

I built `pytest` suites to verify ABAC (Attribute-Based Access Control) to ensure employees could never query documents belonging to other departments.

13. Deployment Constraints

Constraint

Deploying on free-tier Render means ephemeral file systems. Uploaded PDFs vanish on restart.

Engineering Response

I designed the ingestion pipeline to extract text, chunk it, and save it directly to PostgreSQL immediately upon upload. The raw file is discarded. I designed around the constraint rather than assuming unlimited infrastructure.

14. Current State vs What is Not Complete

IMPLEMENTED: FastAPI + React 19 + Supabase pgvector.

IMPLEMENTED: Hybrid RAG, Delegating Multi-Agent Architecture, SQL Knowledge Graph, WebSockets, Regex PII Redaction.

NOT IMPLEMENTED: Reciprocal Rank Fusion (RRF).

NOT IMPLEMENTED: Unrestricted Autonomous Tool-Calling Loops.

NOT VERIFIED: Formal SOC2 / GDPR compliance.

LEGACY: ChromaDB is configured in code but superseded by pgvector.

15. What This Project Taught Me

1. Architecture changes under deployment constraints. In-memory prototypes (NetworkX) rarely survive serverless deployments.
2. Retrieval quality is just as important as LLM quality. A GPT-4 model fed the wrong chunks is useless.
3. Security must be designed into the data flow. Adding PII redaction after chunks are stored is impossible; it must happen during ingestion.
4. AI systems require fallback mechanisms. Building dynamic fallback iterators saved the system from external API deprecations.

16. Final Status Matrix

Vector Store: PostgreSQL pgvector (IMPLEMENTED)
Knowledge Graph: SQL-Backed kg_nodes (IMPLEMENTED)
RAG Retrieval: Hybrid Vector + BM25 (IMPLEMENTED)
RRF: Planned (NOT IMPLEMENTED)
Real-time Updates: WebSockets (IMPLEMENTED)
Security: ABAC + PII + AES Crypto (IMPLEMENTED)
Autonomous Agents: Delegation Routing (PARTIALLY IMPLEMENTED)